How Portable Are LLM-Serving Scheduler Rankings Across Workloads, Operating Regions, and Metrics?

Soroush Vahidi

Draft V2 — Phase-12 RQ1–RQ5 results populated, RQ6 (real-vLLM) pending, 2026-09-02

Abstract

Large language model (LLM) serving systems rely on sophisticated scheduling mechanisms, evaluated on workloads that are themselves heterogeneous across source, time, and operating region. We ask a second-order question: is the *comparative ranking* a scheduler benchmark reports portable across workload source, metric, and operating region? We present the LLM-Serving Scheduler Portability Benchmark (LSSP): a preregistered, paired evaluation spanning three workload sources, 120 windows, a calibrated six-region operating grid, an 11-primary/13-total scheduling-policy panel, and 18,720 completed cells, plus a benchmark sample-complexity study and a targeted real-vLLM validation design. Comparative rankings agree strongly between two of the three sources (Kendall's τ up to 1.0) but are measurably less portable wherever the third source is involved (τ as low as 0.55); 3.6% of pairwise comparisons show practically and statistically supported reversals, concentrated at higher operating load. Real-vLLM validation of the strongest reversal is frozen but not yet executed.

Keywords: LLM inference serving; request scheduling; ranking portability; performance evaluation; workload characterization; benchmark reproducibility

Contents

1 Introduction

Modern LLM serving targets high-performance, GPU-resident inference systems whose scheduling decisions govern how requests share costly accelerator, memory, and batching resources, and serving stacks have grown increasingly sophisticated in how they make those decisions. Beyond simple first-come-first-served execution, modern serving stacks employ continuous batching and paged key-value (KV) cache management [10], iteration-level scheduling [27], chunked prefill to balance prefill and decode work within a batch [1], disaggregated prefill/decode execution across dedicated hardware pools [11, 31], KV-cache-centric disaggregated architectures [12], dynamic instance-level rebalancing [19], explicit fairness objectives [18], output-length-aware admission and memory packing [9], and learning-to-rank decode-priority policies [8]. Each of these mechanisms was introduced, and evaluated, against a particular choice of workload: a specific trace, a specific arrival process, a specific mix of prompt and output lengths, and a specific operating load.

A separate line of work has documented that these workloads are themselves far from uniform. Public production traces from different services and time periods differ substantially in burstiness, request concurrency, prompt and output length distributions, and failure characteristics [11, 21, 24]; large-scale characterization studies of KV-cache reuse and of year-long production traffic report further axes of heterogeneity across clients, models, and time [3, 17]; and dedicated workload-generation frameworks exist precisely because no single fixed trace is thought to be representative of production traffic in general [26]. Individually, prior work has therefore already established two things: that LLM-serving mechanisms are numerous and mechanistically diverse, and that the workloads used to evaluate them are themselves heterogeneous and shift across source, time, and operating region.

What has not been directly asked is a second-order question that follows from combining these two observations. The typical evaluation question in this literature is: *how well does scheduler A perform, relative to scheduler B, on the workload(s) chosen for this paper's experiments?* Given that workloads are heterogeneous, and that a paper's experiments necessarily use only a finite, chosen set of workloads, a comparative claim of this form is only as useful as its *portability*: does the same comparative ordering of schedulers hold if the workload distribution, the operating load, or the evaluation metric is changed to another one a practitioner might reasonably care about? This is a different question from workload heterogeneity itself, and a different question from any single scheduler's sensitivity to load or SLO. It is a question about the reliability of the *comparison*, not about any individual system.

We formalize this as the portability of a comparative scheduler ranking $R(W, M, L)$, where W denotes a workload distribution, window, or source; M denotes an evaluation metric; and L denotes an operating or load region. $R(W, M, L)$ is the ordering (full or partial) that a fixed panel of scheduling policies induces under a fixed benchmark protocol, holding W, M, and L at specified

values. Ranking portability asks how R behaves as W, M, or L is varied: whether the ordering of policies is preserved, where and how it reverses, and how much evidence (how many independent workload windows, drawn from how many sources) is required before a reported ranking can be trusted to generalize beyond the exact windows used to produce it.

This question matters because a scheduler comparison is scientifically and practically useful only insofar as its comparative conclusion transfers to the workload distributions a reader actually cares about. A benchmark result of the form "scheduler A outperforms scheduler B" is not a claim about A and B in the abstract; it is implicitly a claim about A and B *under the workload, metric, and load conditions the benchmark used*. If that comparative ordering is fragile under source, metric, or load-region substitution, then benchmark results risk being read as more general than they are, and evaluation effort risks being spent on a single workload family when the resulting conclusion may not transfer. Conversely, if comparative orderings are portable across independent sources, metrics, and load regions, that is itself a useful, non-obvious finding: it would mean a leaner benchmark protocol is sufficient for future scheduler comparisons, and it would give scheduler designers a much stronger warrant for claims of general improvement.

We refer to the benchmark built around this question as the LLM-Serving Scheduler Portability Benchmark (LSSP). LSSP asks the following research questions, fixed under a preregistered protocol before any comparative outcome was generated (§3):

RQ1. How stable are scheduler rankings across independent workload sources?

RQ2. How stable are scheduler rankings under temporal, provider, and domain shifts?

RQ3. To what extent do rankings obtained on synthetic stress workloads transfer to rankings on independent real-trace-derived workloads?

RQ4. Which source-native observable workload characteristics are associated with cross-distribution scheduler rank reversals?

RQ5. How many independent workload windows are required before a comparative scheduler ranking becomes statistically stable?

RQ6. Do representative simulated scheduler rankings and cross-workload rank reversals reproduce on a real serving engine?

Load-level dependence is deliberately treated as a secondary robustness/sensitivity axis applied to RQ1–RQ4, rather than as a headline research question in its own right, to avoid duplicating a closely related prior load-scaling study on an overlapping window set (§3).

Contributions. Because LSSP's comparative scheduler outcomes had not yet been generated at the time this manuscript was prepared (§3), we state our contributions in a form that does not depend on, and is not falsified by, either possible direction of that future result:

- We formulate the portability of a comparative scheduler ranking, $R(W, M, L)$, as a benchmark object in its own right, distinct from both workload characterization and single-scheduler evaluation.
- We design a paired, multi-source evaluation protocol that holds a fixed panel of scheduling policies constant across independently sourced workload windows, operating regions, and metrics, so that ranking-portability statistics are computed on genuinely paired observations rather than on separately tuned, per-source experiments.

- We assemble a mechanism-diverse, fixed scheduler panel spanning classical, fairness-aware, KV-aware, deadline-aware, and faithfully reimplemented external policies, selected under an outcome-independent selection rule fixed before any pilot result was observed (§5).
- We define an uncertainty and reversal-detection methodology that distinguishes practically meaningful rank reversals from microscopic sign changes, using preregistered practical-margin and confidence-interval criteria, and that treats every completion-conditioned metric's undefined cases explicitly rather than by imputation (§5).
- We define a benchmark sample-complexity methodology that asks how many independent workload windows are required before a reported ranking is likely to reproduce, rather than assuming a conventionally sized window set is sufficient.
- We characterize the workload corpus underlying the benchmark, establishing that the workload sources used are measurably different in observed descriptor distributions and strongly separable under a leakage-resistant, chronologically blocked evaluation (§4).
- We plan a paired public artifact release of the benchmark's frozen workload identities, policy panel, protocol, and (once generated) outcome and analysis artifacts, to support independent reproduction (§11).

The frozen Pilot-V2 protocol has since executed (§11), and Sections 6–7 report its structurally validated results: comparative rankings are neither uniformly stable nor uniformly unstable, but source-pair- and load-region-dependent in a specific, legible pattern (§9). This manuscript does not claim that any scheduler in the panel wins in general, and RQ6 (§8) – whether the strongest identified reversal reproduces on real vLLM/GPU execution – remains a frozen, not-yet-executed result contract, stated without asserting its direction in advance.

2 Related Work

2.1 Serving Architectures and Scheduling Mechanisms

A substantial body of systems work has improved LLM inference throughput and latency through scheduling and memory-management mechanisms. Orca introduced iteration-level scheduling, interleaving requests at the granularity of individual decoding steps rather than whole requests [27]. vLLM introduced PagedAttention, a paged virtual-memory-style KV-cache allocator that substantially raised achievable batch sizes and became a widely adopted serving-system reference point [10]. Sarathi-Serve introduced chunked prefill, splitting long prefill computations across multiple decode-interleaved steps to reduce head-of-line blocking under continuous batching [1]. Splitwise and DistServe independently proposed disaggregating the compute-bound prefill phase and the memory-bound decode phase onto separate hardware pools connected by fast interconnects, each showing throughput or cost improvements from phase-specific resource allocation [11, 31]; Mooncake extends this direction with a KV-cache-centric disaggregated architecture that treats a shared, elastic KV cache – rather than the compute engines themselves – as the central resource to schedule around [12]. Llumnix adds dynamic, cross-instance request migration and rebalancing on top of a serving cluster [19]. DynamoLLM addresses cluster-level energy and cost efficiency by dynamically reconfiguring an inference cluster's resource allocation under latency SLOs [21].

A second cluster of work targets scheduling *policy* design more specifically. VTC (Virtual Token Counter) defines an explicit token-level fairness objective for continuous-batching schedulers

and proves a bound on the resulting service-difference guarantee [18]. S^3 predicts each request's output length ahead of generation and uses that prediction to pack KV-cache memory more tightly at admission time, trading occasional misprediction-driven preemption for higher achievable batch size [9]. Fu et al. [8] instead learns a pairwise ranking model over waiting requests to prioritize decode order – a distinct mechanism from S^3's admission-time length prediction, despite both papers appearing at NeurIPS in consecutive years and both targeting throughput under uncertain output length. FastServe, JITServe, PARS, Libra, SLAI/RAD, SCORPIO, LMetric, Apt-Serve, and ProServe extend this space further with iteration-level preemption, SLO-aware admission under imprecise request information, pairwise learning-to-rank variants, dynamic request partitioning, theoretically motivated multi-objective scheduling, heterogeneous-SLO-aware admission, lightweight scheduling metrics, adaptive hybrid-cache scheduling, and unified multi-priority scheduling, respectively [4, 13, 14, 16, 22, 23, 25, 29, 30]. Each of these papers reports an improvement over one or more baselines on the workload(s) chosen for its own experiments; none of them, individually or collectively, asks whether the *relative ordering* they report would be preserved under an independently sourced workload, a different metric, or a different operating load – the question this benchmark is built around (§2.7).

2.2 Production Workloads and Characterization

A parallel literature documents that LLM-serving workloads observed in production are themselves heterogeneous. BurstGPT releases 10.31 million traces collected from a regional deployment of Microsoft's Azure OpenAI GPT service over 213 days, characterizing request burstiness, conversation structure, and response-length behavior, and is now published as a full KDD 2025 paper rather than only as a preprint [24]. Two official Microsoft releases document Azure's own internal LLM-inference traffic: Splitwise analyzes production conversation and code-completion traces collected on Azure in November 2023 [11], and DynamoLLM analyzes a longer, one-week Azure trace collected in May 2024 [21]. anonymized per venue record; see primary source [3] characterizes KV-cache reuse patterns at a large cloud provider, finding that reuse probability and lifespan are workload-category-dependent in ways that synthetic workload studies had not captured. see primary source for full author list [17] reports a one-year longitudinal characterization of production traffic spanning workload evolution, caching behavior, and load balancing (a recent preprint at the time of writing; §2.7). ServeGen instead characterizes a worldwide production inference service in order to build a principled per-client workload *generator*, motivated explicitly by the position that no single fixed trace is representative enough on its own [26]; because it characterizes the same underlying Alibaba Cloud platform as this project's Bailian/Qwen source, we do not count it as an independent workload provider (§4). TraceLab characterizes a distinct workload family entirely – coding-agent sessions rather than single-turn or short-multi-turn chat requests – and is used here only as out-of-distribution context, not as one of this project's primary Pilot-V2 sources.

2.3 Simulators and Benchmark Frameworks

Vidur is the closest existing infrastructure precedent to LSSP: a large-scale simulation framework purpose-built for exploring LLM-inference system configurations at a fidelity and speed unavailable from running full experiments on real hardware [2]. LLMServingSim and its successor LLMServingSim 2.0 provide a related but architecturally different hardware/software co-simulation infrastructure, adding iteration-granularity simulation with algorithmic-redundancy reuse and, in the 2.0 revision, explicit support for heterogeneous accelerators and disaggregated serving techniques [6, 15]. These simulators are built to let researchers explore serving-system design points cheaply;

none of them is built around, or reports, a benchmark of *comparative scheduler ranking portability* across independently sourced workloads in the sense this paper defines. LSSP's own simulator (§5) is project-local infrastructure, not a contribution positioned against this literature; we make no claim that its output is equivalent to production hardware latency or throughput, only that it supports a fixed, controlled benchmark protocol for relative comparisons (§5).

2.4 Workload Dependence and Benchmark Sensitivity

Prior work has already established, individually, that LLM-serving workloads are heterogeneous (§2.2) and that individual serving mechanisms and schedulers are sensitive to batching structure, prefill/decode balance, load level, and service-level objectives (§2.1). We treat both of these as accepted background, not as a novel contribution of this work. What we do not find in the literature reviewed here is a direct study of whether a *comparative ordering* among a fixed panel of schedulers is itself portable across independently sourced workloads, operating regions, and metrics – the distinction this paper's Introduction develops between single-system sensitivity and comparative-ranking portability.

2.5 Ranking Portability and Benchmark Methodology

The statistical methodology this benchmark uses – Kendall's tau-b and Spearman's rho for pairwise ranking agreement, paired comparison and block-bootstrap confidence intervals for ranking-uncertainty quantification, and a probability-of-correct-selection framing for benchmark sample complexity – draws on standard nonparametric statistics and comparative-algorithm-ranking methodology (§5); these are general statistical tools, not literature claims specific to LLM serving, and are cited here only as the methods this benchmark's analysis plan applies to its own, independently generated data. The closest methodological precedent we identified is from the classical parallel-job- scheduling literature: Zakay and Feitelson [28] partition a real scheduler workload trace into independent per-user subtraces and recombine them to resample varied-but-realistic workloads, evaluating how a *single* parallel job scheduler's measured performance responds to workload resampling. LSSP's sample-complexity question (§7) is the same evaluation instinct – using resampling to ask how much workload evidence a scheduler-performance conclusion actually rests on – applied to a different object: the portability of a *comparative* ranking among multiple schedulers across independently sourced workloads, rather than one scheduler's sensitivity to resampled variants of a single trace.

2.6 Recent *Journal of Supercomputing* Context

Three recent *Journal of Supercomputing* papers illustrate the journal's current engagement with GPU-resident LLM-inference performance and portability, at layers adjacent to but distinct from this benchmark's request-scheduling focus. Doosthosseini et al. [7] address deploying LLM inference services on Slurm-managed HPC clusters without compromising existing cluster security policy – a *cluster job-placement* scheduling problem, one level above the *intra-server request* scheduling this benchmark studies. Carmona-Martínez et al. [5] and Siwinska et al. [20] each study a different sense of "portability" for LLM/transformer workloads on GPU (and, in the latter case, TPU) hardware: compiler-backend selection trade-offs and auto-tuning-framework portability across accelerator hardware, respectively – portability of *implementation performance across hardware*, not portability of a *comparative ranking across workloads* at fixed hardware, which is this benchmark's object of study. We cite these three papers to situate LSSP within the journal's live interest in

LLM-serving and GPU-inference performance, not because they address this benchmark's specific research question.

2.7 Position of LSSP

LSSP asks a different question than the majority of the prior work surveyed above. Prior work generally studies a scheduler, a serving system, a simulation environment, or a workload generator/characterization in its own right. LSSP instead studies the portability of *comparative* scheduler rankings across independently sourced workload families, operating regimes, and metrics, using a fixed, mechanism-diverse scheduler panel and a preregistered protocol. To our knowledge, the LSSP benchmark is distinct in treating the portability of comparative scheduler rankings itself as the primary object of study across independent workload sources, operating regimes, and metrics, while explicitly quantifying ranking uncertainty and benchmark sample complexity. A literature search targeting exactly this question (scheduler-ranking stability/portability, cross-workload scheduler evaluation, benchmark sample complexity and representativeness for LLM-serving systems, current through September 2026) did not surface prior work that treats comparative scheduler-ranking portability itself as its primary object across independent LLM-serving workload sources; the closest single-scheduler-focused methodological precedent (Zakay and Feitelson [28], §2.5) predates the LLM-serving literature entirely and evaluates one scheduler's sensitivity to workload resampling, not comparative ranking across schedulers. Vidur is the closest infrastructure precedent (§2.3); LLMServingSim and its successor are related but differently scoped simulator infrastructure; the recent production-characterization literature (§2.2) establishes the workload heterogeneity this benchmark's design responds to, but does not itself study cross-source ranking portability. This wording is intentionally narrower than an unconditional claim of priority; we do not claim to be the first to study LLM-serving workload dependence or scheduler sensitivity, only that we did not identify a prior benchmark treating cross-source, cross-metric, cross-load-region scheduler-ranking portability as its primary object of study under the primary-source review conducted for this manuscript.

A note on provider independence is warranted here because it bears directly on how several of the sources above should be read together (§4 gives the full treatment). BurstGPT's own reported methodology collects its traces from a regional customer's use of Microsoft's Azure OpenAI GPT service [24]; this project's own Azure-sourced traces are Microsoft's own official releases of internal Azure LLM-inference-service traffic [11, 21]. Both therefore sit on the same underlying cloud provider's infrastructure, even though they are collected by different teams through different mechanisms and released as distinct, independently versioned artifacts. We describe BurstGPT and this project's Azure traces as distinct, independently released workload sources throughout this manuscript, and we do not describe them as independent providers.

3 Study Design

3.1 Research Questions

We reproduce the project's frozen research questions verbatim (see `docs/RESEARCH_QUESTIONS.md` for the full changelog):

RQ1. How stable are scheduler rankings across independent workload sources?

RQ2. How stable are scheduler rankings under temporal, provider, and domain shifts?

RQ3. To what extent do rankings obtained on synthetic stress workloads transfer to rankings on independent real-trace-derived workloads?

RQ4. Which source-native observable workload characteristics are associated with cross-distribution scheduler rank reversals?

RQ5. How many independent workload windows are required before a comparative scheduler ranking becomes statistically stable?

RQ6. Do representative simulated scheduler rankings and cross-workload rank reversals reproduce on a real serving engine?

Load-level dependence is a secondary robustness/sensitivity analysis applied to RQ1–RQ4's findings, not a headline research question (§3.3).

3.2 Study Overview

LSSP evaluates a fixed panel of 13 scheduling policies (§5.1) against a frozen corpus of 120 workload windows drawn from three independently sourced traces (§4.1), across a calibrated six-region grid of operating regions (§5.5) and a fixed set of primary and completion-conditioned metrics (§5.6). Every policy is run against every window under every operating region, with a deterministic rep0/rep1 verification pass (§5.7), producing $120 \times 6 \times 13 \times 2 = 18{,}720$ completed, independently validated cells, from which per-source, per-region, and per-metric scheduler rankings are derived. Ranking-portability statistics (Kendall's tau-b, Spearman's rho, top-k overlap, and pairwise reversal detection) are then computed across pairs of sources, regions, and metrics, using workload window as the statistical unit (§3.4). This design was frozen (§3.5) before any comparative scheduler outcome existed.

3.3 Evidence Independence / Publication Boundaries

This project shares a lineage and a simulator codebase with two other, separately scoped manuscripts, and its design was explicitly audited for overlap against both before this study's protocol was frozen (`docs/OVERLAP_LEDGER.md`, `docs/GO_NO_GO_GATES.md` Gate A, `docs/EVIDENCE_INDEPENDENCE_PLAN.md`):

- **"The Exploitability Gap in LLM-Serving Scheduler Portfolios" (LLM 2026).** That manuscript's `public_replay_load_scaling_v1/v2` experiment used a fixed set of 60 canonical public-trace windows (20 BurstGPT + 20 Azure-2023 conversation + 20 Azure-2023 code), a load-factor grid $\{1, 2, 4, 8, 16, 32, 64, 128\}$, an 8-policy portfolio, and ANWG/SBS-VBS exploitability diagnostics. That result is treated here as `PRIOR_RESULT_REFERENCE_ONLY`: citable as prior motivation, never restated as this project's own evidence. This project's own window sampling is independently constructed (§4.3) and verified non-identical to that 60-window set; load-level dependence is accordingly demoted to a secondary analysis (§3.1) using this project's own calibration protocol (§5.5), never that load grid.
- **"Module Counterfactuals for Attributing LLM Serving Scheduler Effects" (a separate module-intervention manuscript).** This study makes no module-counterfactual or module-attribution claim; RQ4's workload-descriptor analysis (§5) is an offline, descriptive association analysis, not a deployed selector or an intervention-based attribution method.
- Other project lineage (an unrelated database/systems research thread) is unconnected to this manuscript's content and is not referenced here.

3.4 Experimental Unit and Paired Design

The statistical unit of analysis is the workload window: a fixed-size (200-request) contiguous slice of a source trace, uniquely identified and frozen before any scheduler outcome existed (§4.3). Every policy in the panel (§5.1) is run against every window under every calibrated operating region (§5.5), so that ranking-portability statistics are computed on genuinely paired observations – the same window, region, and metric definition, varying only the policy – rather than on separately tuned per-source experiments. Source, operating region, and metric are treated as factors of interest; policy identity is the object whose relative ordering is measured.

3.5 Preregistration

The workload window set (120 windows: 40 per source, 10 Stage-0-reused + 30 Pilot-V2-new per source; §4.3), the scheduler panel and its selection rule (§5.1), the metric contract (§5.6), the six-region operating-load calibration (§5.5), and the ranking-portability analysis plan (reversal-detection thresholds, bootstrap procedure, sample-complexity design) were each frozen, and cross-checked against a structural validation gate, before any Pilot-V2 comparative scheduler outcome was generated. `docs/RANKING_PORTABILITY_PILOT_V2_PROTOCOL.md`, `docs/RANKING_PORTABILITY_ANALYSIS_PLAN.md`, and `docs/RANKING_PORTABILITY_WINDOW_FREEZE.md` record the exact frozen content and the gate checks each passed.

4 Workload Corpus and Characterization

4.1 Sources

LSSP's primary Pilot-V2 corpus draws from three independently released workload sources: BurstGPT [24], Microsoft's official Azure 2024 LLM-inference trace [21], and a Bailian/Qwen production trace release. Azure's 2023 conversation/code traces [11] and Stage-0's historical windows contribute additional context but are not part of the primary Pilot-V2 120-window matrix beyond the 10-per-source Stage-0-reused windows described below. TraceLab [32] is retained as an out-of-distribution, coding-agent workload family for future expansion, not as one of the three primary sources, because its independence from a prior 512-window public sweep cannot be fully verified from public artifacts alone (`docs/TRACELAB_PROVENANCE_RESOLUTION.md`).

As established in §2.7, BurstGPT's own reported collection methodology logs traffic from a regional customer of Microsoft's Azure OpenAI GPT service, and this project's Azure trace is Microsoft's own official release of internal Azure LLM-inference-service traffic. Both therefore sit on the same underlying cloud provider's infrastructure. We describe BurstGPT and Azure as distinct, independently released workload sources – different collection methodology, different customer population, different release provenance and version history – and we do not describe them as independent providers. Bailian/Qwen (Alibaba Cloud) is a genuinely different underlying platform from both.

4.2 Representation and Provenance

Each source is ingested through a dedicated, field-level-audited adapter (`docs/DATA_FIELD_PROVENANCE.md`) that maps the source's native schema onto a common internal request representation (arrival time, input tokens, output tokens, total tokens, session identifier where available, model class). No field is fabricated where a source's native schema lacks it (for example, BurstGPT's documented schema has no tenant, KV-reuse, SLO, or failure-code field, and none is synthesized).

License status for each source is recorded in `docs/DATA_LICENSE_AUDIT.md` (BurstGPT: CC-BY-4.0) and re-verified before any confirmatory use of a real, full-scale sample.

4.3 Window Construction

The Pilot-V2 corpus freezes exactly 120 windows of 200 requests each: 40 per source (`azure_llm_2024`, `bailian_qwen`, `burstgpt`), each composed of 10 windows reused verbatim from the Stage-0 pilot (§5.4) plus 30 new windows drawn by a deterministic, outcome-blind, source-symmetric pinned-extension sampler that uses only valid-row index ranges and fixed seeds – never scheduler metrics, policy identities, or Stage-0 tie/non-tie outcomes (`BURSTGPT_OUTCOME_TARGETING = NO`, structurally verified). Windows are assigned to EARLY/MIDDLE/LATE chronology strata by deterministic relative chronology (60/30/30 windows respectively); Stage-0-reused windows are all EARLY. The freeze passed a full structural validation gate – unique window identities, exact per-source and Stage-0/new counts, fixed `request_count = 200`, no prohibited overlap, valid chronology, and absence of any scheduler/policy/load-calibration outcome field in the selection rule – recorded with a canonical content hash (excluding generation timestamp; full value in Table 3) before any Pilot-V2 load calibration or scheduler outcome was generated.

4.4 Descriptors

Each window is described by a 28-feature descriptor vector spanning prompt- and output-token-length statistics (mean, p90, long-prompt fraction at the 512/2048/8192-token thresholds), arrival-timing statistics (mean arrival rate, inter-arrival percentiles), and derived load-pressure proxies. Three deterministically redundant pressure-proxy features ($R^2 = 1.0$ against other included features) are retained in the underlying dataset but excluded from the classifier input used in §4.6. Prompt-token figures compose differently across sources for architectural reasons: TraceLab's and Azure's prompt-token fields are potentially multi-turn-cumulative context, while BurstGPT's is a single per-request figure. This is a property of how each source's underlying workload is architected (a growing agent/conversation session versus a single stateless API call), not an adapter artifact, and it is preserved as an explicit caveat on every prompt-length-based claim below.

4.5 Cross-Source Differences

Under a leave-one-time-bucket-out (EARLY/MIDDLE/LATE), grouped, chronologically blocked cross-validation over $n = 400$ windows (window size 200), a random forest classifier distinguishes the four descriptor-labeled source classes with pooled balanced accuracy 1.000 (mean ± std across three folds: 1.000±0.000); a multinomial logistic regression reaches 0.988 balanced accuracy (0.987±0.007 across folds); and a depth-limited (depth ≤ 3) decision tree reaches 0.980 balanced accuracy (0.980± 0.018 across folds, worst single fold 0.955 with LATE held out). This grouped, chronologically blocked evaluation supersedes an earlier random-split result (which agreed at balanced accuracy 1.0) for any manuscript claim, because random splitting is vulnerable to within-window-adjacent-time leakage that the grouped evaluation is structurally immune to (regression tested; `tests/test_separability_pipeline_no_leakage.py`).

4.6 Source Separability

We report the final characterization result as `SEPARABILITY_RESULT_CONFIRMED_WITH_CAVEATS`. The safe claim supported by this evidence is that the workload sources' observed descriptor distributions are measurably different, and that source identity is strongly separable under a leakage-

resistant, chronologically blocked evaluation. This is not evidence that the underlying providers' conversations or tasks intrinsically differ in length or structure, only that their *observed, request-level* descriptor figures do, for a mixture of architectural and possibly-genuine-content reasons that this classifier-based analysis cannot separate (§4.4).

4.7 Drivers of Differences

An attribution analysis triangulating held-out permutation importance, single-feature balanced accuracy, leave-one-feature-out deltas, and feature-group-ablation deltas (all under the same grouped chronological cross-validation) identifies two tiers of driving features. Tier 1 – incrementally load-bearing, with nonzero permutation importance and a measurable accuracy cost when the `prompt_length` feature group is removed (-0.0051) – consists of `long_prompt_fraction_512`, `long_prompt_fraction_2048`, and `prompt_tokens_mean`. Tier 2 – individually strong single-feature classifiers (balanced accuracy up to 0.980) whose removal, individually or as a whole feature group, costs 0.000 accuracy because their signal is redundant with Tier 1 – includes total/prompt-token percentile features, inter-arrival percentiles, and mean arrival rate. Prompt-length structure, specifically the fraction of long prompts at the 512- and 2048-token thresholds and mean prompt length, is therefore the primary, non-redundant driver of source separability; arrival-timing structure and output length are individually predictive but carry no additional signal once prompt-length features are present. A critical-ablation check confirms this separability is broad rather than dependent on any single feature: removing the two strongest Tier-1 features together changes random-forest balanced accuracy by at most -0.0025.

4.8 Window-Size Sensitivity

The Pilot-V2 corpus uses a fixed window size of 200 requests throughout, matching Stage-0 and the LLM 2026 comparison window size for methodological consistency. A dedicated window-size sensitivity sweep (varying window size and re-checking separability/discriminability) has not yet been run; this is noted as an open item in §10 rather than a result reported here.

5 Scheduler Benchmark Methodology

5.1 Scheduler Panel

The PRIMARY panel comprises 11 policies, executed alongside 2 additional `STYLE_APPROXIMATION` policies used only for a mandatory robustness check (13 policies executed in total): a classical arrival-order baseline (`fifo`, also the load-calibration reference policy), a deadline-aware policy (`edf`), a slack/urgency-aware policy (`least_laxity_first`), a service-length-aware policy (`estimated_service_time_first`), a fairness-aware policy (`weighted_fair_share`), a KV/memory-pressure-aware policy (`kv_constrained_online`), an admission-control/SLO-guard policy (`admission_control`), and faithful reimplementations of four external mechanisms: vLLM-style token-budget continuous batching in both its pre-chunked-prefill and chunked-prefill forms (`vllm_faithful`, `vllm_chunked_prefill_faithful`), Sarathi-style chunked prefill (`sarathi_faithful`), and an SLO-aware scheduling policy in the SLAI/RAD family (`slai_faithful`). The two `STYLE_APPROXIMATION` robustness policies are a token-budget-proxy style approximation (`vllm_style_token_budget`) and an SLO-guard-proxy style approximation (`scorpio_style_slo_guard`); two further faithful reimplementations requiring disaggregated/cross-instance simulator semantics (`distserve_faithful`, `llumnix_faithful`) are executed only in a secondary

stratum, never pooled with the single-GPU colocated PRIMARY leaderboard, and one candidate (`apt_serve_faithful`) is excluded entirely as `scaffolding_only` – not a validated reimplementation at the time of this freeze.

A policy enters the PRIMARY panel only if, at the time it was added to the comparability audit (frozen the day before Stage 0 executed), it satisfies an outcome-independent selection rule: its mechanism predates this protocol's freeze, it occupies a mechanism family not already saturated by an existing panel member, it has adequate implementation fidelity and passes its own unit tests, it is simulator-semantically compatible with the single-GPU colocated execution model, and it is independently reproducible from a cited reference. No policy was added to or removed from the panel in response to any Stage-0 outcome (§5.4); the panel document predates Stage-0 execution by a full day, and mechanism families identified as under-implemented (preemptive/multilevel scheduling in the FastServe sense, JITServe, LMetric) were deliberately left as documented future-work candidates rather than fast-tracked into the panel after seeing Stage-0's BurstGPT collapse.

5.2 Fidelity Taxonomy

Every executed policy is labeled with one of five fidelity classes: `FAITHFUL_EXTERNAL` (a validated reimplementation of a specific published mechanism), `REPOSITORY_NATIVE_CLASSICAL` (a standard scheduling discipline implemented natively in this simulator, not claimed as a reproduction of any specific paper), `SIMULATOR_PROXY` (a mechanism-motivated policy without a single canonical external paper), `STYLE_APPROXIMATION` (an approximate, style-level proxy for a published mechanism, explicitly excluded from the PRIMARY headline panel and used only for robustness checks), and `scaffolding_only` (an unaudited, not-yet-validated implementation, excluded entirely). We do not claim that any `STYLE_APPROXIMATION` or `SIMULATOR_PROXY` policy is an official or validated reimplementation of its namesake published system.

5.3 Execution Model

Each (window, policy, operating-region) cell is executed as a fully deterministic discrete-event simulation: given identical inputs, the simulator produces bit-identical outputs, with no consumed randomness (verified by a repeated-run determinism test). This determinism is what permits the paired design of §3.4: two policies' outcomes on the same window and region differ only because of the policy's own scheduling decisions, not because of independent sampling noise.

5.4 Stage-0 Discriminability Pilot (Historical)

Before the Pilot-V2 protocol described in the remainder of this section was frozen, a preregistered discriminability pilot ("Stage 0") executed a real, 1,080-cell matrix (array 1213964; three sources, three load regions, a six-policy PRIMARY subset) to check, before committing to the full protocol, that the panel would yield non-trivial scheduler differentiation under a fixed load protocol without post-hoc cherry-picking. **This is historical pilot evidence, not the confirmatory Pilot-V2 result**, and it never measured cross-source ranking portability – that question was and remains explicitly out of Stage-0's scope. Stage 0's own five-criterion discriminability analyzer returned four passing criteria (77.8% of (source, window, load-region) triples non-tied, 86.7% of windows non-tied, no universal policy collapse, and 100% of non-tied cells showing meaningful metric variation) and one genuine failure: BurstGPT contributed only 14.3% of non-tied conditions, against a 15% floor, versus ∼42.9% each for the other two sources – a result reported as `STAGE0_NO_GO`, robust to the completion-conditioned-metric zero-completion convention used. A follow-up diagnostic traced this to a mechanism-level explanation: differentiation was strongly source-dependent, attributable

to a near-total collapse of five of the six Stage-0 policies into pairwise bit-identical outcomes on BurstGPT's structurally short-prompt, near-constant-output traffic – not to BurstGPT receiving weaker or fewer calibrated load conditions than the other sources. This motivated the Pilot-V2 protocol's larger, mechanism-diverse 13-policy panel and denser six-region load grid (§5.5), applied symmetrically to every source, never to BurstGPT alone. Detailed Stage-0 mechanism analysis is deferred to an appendix; it is not restated here as a ranking-portability finding.

5.5 Operating-Region Calibration

A FIFO-only calibration pass establishes six operating regions – `LOW` (load factor 0.5), `PRE_KNEE` (0.8), `KNEE` (1.0), `POST_KNEE` (1.1), `OVERLOAD` (1.2), and `HIGH_PRESSURE` (1.5) – across all 120 frozen windows (720 calibration cells total; 720 valid, deterministic, 0 failed/missing/ duplicated). We report this as calibration provenance, not a comparative scheduler result: FIFO is the calibration reference policy, and the calibration's role is to fix load-factor-to-region mappings before any other policy is executed against them, not to characterize FIFO's relative performance. As a calibration characteristic worth preserving, 301 of the 720 calibration-region records (93 BurstGPT, 101 Bailian/Qwen, 107 Azure) recorded zero completions under FIFO at that region's load factor; this is a property of the calibration design at those specific (window, region) combinations, the frozen calibration validity gate passed regardless (`PHASE11_CALIBRATION_VALID = YES`), completion-conditioned metrics are simply undefined (not zero) in those records (§5.6), and no calibration redesign was performed after observing this count.

5.6 Metrics

Three metrics are always defined for every completed cell: completion fraction, arrival-normalized weighted goodput (ANWG, the primary metric), and weighted completion fraction. A second group of metrics is defined only conditional on at least one request completing: SLO-violation rate, weighted goodput, mean/p95/p99 latency, and request/token throughput are each `NaN` if and only if completion fraction is exactly zero for that cell. Mean/p95 time-to-first-token is subject to a stricter, separately checked precondition (at least one completed request recorded a first-token time), because TTFT can be undefined even when completion fraction is positive. No undefined conditional-metric value is ever imputed as 0 or 1, in either direction, and never after inspecting which policy or source it would affect. Where a policy's value is undefined for a given (window, region, metric) condition, that policy is excluded from the ranking-derived statistic (Kendall's tau-b, Spearman's rho, top-k overlap, pairwise reversal detection) for that specific condition and metric only – never scored as a loss or a tie, and never causing the whole condition to be dropped for policies that did complete. Every ranking-derived table reports its exclusion count (`n_conditions_excluded_for_undefined_metric`) alongside the statistic itself, since a source-concentrated exclusion rate is itself a reportable finding under RQ4, not a footnote.

5.7 Paired Design and Repetitions

Because the simulator is fully deterministic (§5.3), a single execution per (window, policy, region) cell is sufficient; ranking-uncertainty quantification (§6) instead comes from resampling over the independent workload windows themselves (block bootstrap), not from repeated stochastic runs of the same cell.

5.8 Mechanism Telemetry

Every executed cell additionally records a seven-field mechanism-activation telemetry summary: queue-depth (mean/max), batch-saturation (mean/max fraction of `max_active_sequences` in use), prefill/decode contention fraction, KV-occupancy (mean/max fraction of `max_kv_tokens` in use), admission-control activation count, preemption/reorder event count, and token-budget saturation fraction. Each field is always defined: a value of zero means either that the corresponding mechanism is genuinely absent for that policy (for example, preemption count is exactly zero for every policy that never issues a preempt/swap/migrate action, which is most of the panel) or that the underlying simulator mode does not implement a separate prefill/decode phase or token-budget model – not an instrumentation failure. This telemetry is collected for every cell but is not used to support any mechanism-level finding in this manuscript; RQ4's telemetry-to-reversal association analysis (§6) is pre-specified and will only be run once Pilot-V2 outcomes exist.

5.9 Ranking-Uncertainty Quantification and Reversal Classification

Because the statistical unit of analysis is the workload window (§3.4), ranking-uncertainty is quantified by a whole-window block bootstrap: windows, not individual requests, are resampled with replacement (preserving each window's internal request sequence intact), 2,000 resamples are drawn per condition, and every reported confidence interval is a 95% percentile interval over those resamples, using a deterministic, recorded seed so every interval is exactly reproducible from the frozen outcome matrix alone. This block structure exists because the requests within a window are not independent observations of scheduler behavior – they share one arrival process, one prompt/output-length mixture, and one operating region – so resampling at the request level would understate uncertainty; the window is the unit that is actually drawn independently by §4.3's sampling design, and it is therefore the unit that must be resampled.

Two further tests calibrate the reversal statistics reported in §6.3 against multiple comparisons. A Friedman rank-sum omnibus test is applied per (source, region, metric) condition to ask whether the panel's policies differ at all before any pairwise reversal is interpreted, so that pairwise reversal claims are read in the context of whether an omnibus difference exists in the first place. Because the panel and the (source, region, metric) grid together generate many pairwise comparisons, every pairwise reversal p-value is corrected across that condition's family using Benjamini–Hochberg false-discovery-rate control at $q = 0.05$ before a reversal is classified as statistically supported; an uncorrected pairwise p-value alone is never sufficient for the `SUPPORTED_PRACTICAL_REVERSAL` class defined below.

A numerical sign change in a point estimate is not, by itself, a meaningful scheduler reversal from a systems standpoint: two policies can swap point-estimate order by a magnitude far smaller than the benchmark's own measurement noise, in which case treating it as a reversal would overstate what the benchmark actually supports. Every pairwise (policy A, policy B) comparison at a given (source, region, metric) condition is therefore classified into exactly one of five preregistered classes, combining the practical-margin test with the bootstrap/FDR-corrected statistical test:

`UNDEFINED_UNESTIMABLE` At least one of the two policies' metric values is undefined for this condition (§5.6); no direction is estimated or claimed.

`STABLE_NO_SIGN_CHANGE` The two policies' relative order agrees across the compared conditions; no reversal occurred.

`MICROSCOPIC_SIGN_CHANGE` The point-estimate order reverses, but the winning margin never exceeds 10% of the losing policy's value in either condition – a numerically real but practically

negligible sign flip, reported separately from, and never pooled with, the classes below.

`UNSUPPORTED_SIGN_CHANGE_WIDE_CI` The point-estimate order reverses with a practically meaningful margin in at least one condition, but the FDR-corrected 95% bootstrap interval for the difference does not exclude zero in both conditions – the apparent reversal is not statistically distinguishable from bootstrap noise at the preregistered confidence level.

`SUPPORTED_PRACTICAL_REVERSAL` The point-estimate order reverses *and* the winning margin exceeds 10% of the losing policy's value in both compared conditions *and* the FDR-corrected 95% bootstrap interval for the direction excludes zero in both conditions. Only this class is reported as a headline reversal in §6.3; the other four classes are always reported alongside it, never omitted as a residual category.

5.10 Temporal Analysis Design

Temporal portability is evaluated separately per source, using each source's own native chronology rather than a single shared calendar rule, because the three sources differ in what temporal structure they actually expose (§4.1). BurstGPT windows are assigned to EARLY/MIDDLE/ LATE chronology terciles by deterministic relative order within the trace (§4.3; 60/30/30 windows respectively), and a secondary EARLY/LATE bisect split is run as a robustness check on whether the tercile boundary choice itself drives any temporal finding (§5.11). Azure 2024's temporal split instead uses the trace's own absolute calendar timestamp, frozen at Unix epoch `1715731200.0` (2024-05-15T00:00:00Z), splitting the trace into before/after halves at that boundary; this is a genuine calendar split, not a relative-order tercile, because Azure's release exposes an absolute timestamp field that BurstGPT's schema does not (§4.2). Bailian/Qwen's release does not expose a verified absolute timestamp field usable for a calendar split; its temporal analysis is therefore restricted to a relative-order comparison only, explicitly labeled `RELATIVE_CHRONOLOGY_ONLY` in every table it appears in, and is never presented alongside BurstGPT's tercile result or Azure's calendar-split result as though the three used an equivalent temporal definition (§6.4). No cross-source temporal number is computed by pooling these three distinct temporal definitions into one figure.

5.11 Robustness Analyses

Every headline ranking-portability result (§6) is pre-specified to be checked against a fixed set of robustness analyses: the 11-policy high-fidelity subset (PRIMARY panel excluding both `STYLE_APPROXIMATION` policies), leave-one-source-out re-estimation, window-size sensitivity, an alternate completion-conditioned-metric convention, the full six-region load grid, a temporal-split sensitivity check (§5.10's BurstGPT bisect), and mechanism-family-exclusion checks. These robustness analyses are part of the frozen protocol; results for the six actually executed and materialized by the sealed analysis code are reported in §6.5 (one, `METRIC_DEFINITION_SENSITIVITY`, has no implementing code in the sealed launcher despite its status record and is noted there as a disclosed open item).

5.12 SLO-Definition Sensitivity: A Disclosed Protocol Gap

An SLO-definition sensitivity check – re-running the benchmark's deadline-dependent policies under an alternative SLO-synthesis rule to confirm that any headline reversal is not an artifact of the one synthesis choice used to fabricate `slo_deadline` where a source does not natively carry it (§4.2)

– was planned as part of this protocol. We disclose here, as a matter of methodological discipline rather than as a result, that no alternative SLO-synthesis rule was ever actually frozen before or during Pilot-V2 execution: every document that references "the alternative SLO-synthesis rule" points to another document as the place it is defined, and that chain does not terminate in an actual formula, multiplier, or distribution anywhere in this project's history. Consequently: no alternative rule was invented after Pilot-V2 execution began; no SLO-sensitivity experiment has been run; this robustness component is *not* claimed as preregistered evidence anywhere in this manuscript; and any future SLO-sensitivity study must be explicitly labeled a post-campaign robustness extension, with its own freeze and manifest process, unless a genuinely prior external protocol specifying the alternative rule is located.

6 Scheduler Ranking Portability

The results below are computed from the six canonical Phase-12 analysis artifacts, generated by the sealed analysis code (`eb574a8ce5c34a80fddbcfd4417f6626fbdddfd1`) against the admitted, independently re-validated 18,720-cell consolidated matrix, and structurally audited before interpretation (§11). Reported values are computed deterministically by `paper/scripts/generate_phase12_tables_figures.py` from those six hash-verified artifacts; no number below was entered by hand. The PRIMARY headline metric throughout is arrival-normalized weighted goodput (ANWG, §5.6); other metrics are reported as secondary/supporting evidence only. Two subsections of the original result contract – per-source *discriminability* (fraction of non-tied conditions) and cross-*metric* rank agreement – are not part of this analysis run's six canonical artifacts and remain open items, noted where relevant rather than populated with placeholder or approximated numbers.

6.1 Cross-Source Rankings

Across the 3 source pairs × 6 operating regions (18 conditions), Kendall's tau-b for the PRIMARY metric ranges from 0.55 to 1.00, with a per-region mean between 0.69 and 0.77 (Table 1, Figure 1). Every one of the 18 conditions has a positive, non-trivial tau-b: comparative scheduler rankings are never uncorrelated or inverted *in aggregate* between any two sources at any region. Portability is, however, visibly uneven across source *pairs* rather than uniform: Azure-2024 and Bailian/Qwen agree very strongly with each other at every region ($\tau = 0.89$–1.00), while every comparison involving BurstGPT is substantially lower ($\tau = 0.55$–0.71) – a gap of roughly 0.2–0.4 in Kendall's tau-b that holds at all six regions, not just one. Top-1 agreement (the single best-performing PRIMARY-panel policy) and top-3 agreement follow the same source-pair pattern in the underlying comparisons table (`paper/generated/table_data/rq1_rq2_portability.json`); we do not tabulate top-1/top-3 separately in the main text to avoid restating the same qualitative pattern three times, but the full table is part of the planned artifact release (§11).

Table 1: Primary-metric (ANWG) Kendall's tau-b, mean across the three source pairs, per operating region.

	LOW	PRE_KNEE	KNEE	POST_KNEE	OVERLOAD	HIGH_PRESSURE
mean τ	0.69	0.77	0.74	0.74	0.72	0.75
min τ	0.55	0.70	0.60	0.61	0.58	0.62
max τ	0.92	0.89	0.96	0.96	0.96	1.00

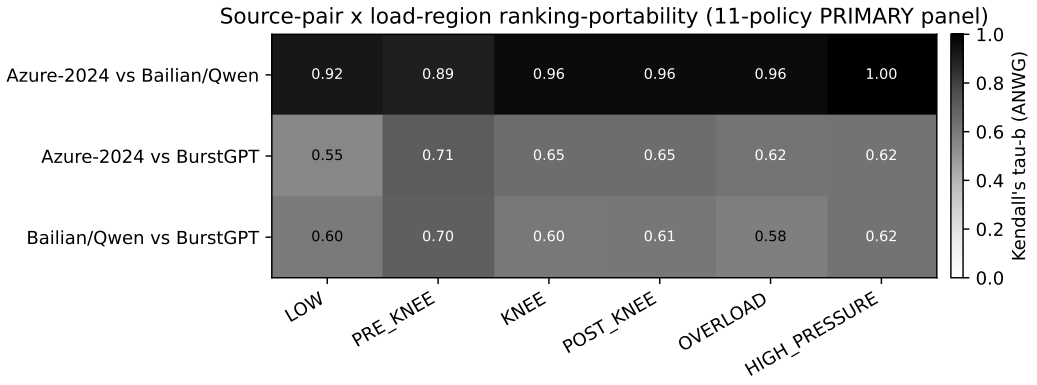

Figure 1: Source-pair × load-region ranking-portability: Kendall's tau-b between each pair of sources' PRIMARY-panel (11-policy) rankings on the ANWG metric, at each of the six frozen operating regions (§5.5). Computed from `ranking_correlations.json` (SHA-256 `d77d5e97…`, full value in `paper/generated/table_data/rq1_rq2_portability.json`).

The min/max range in Table 1 makes clear that portability is not homogeneous even within a region: the range at LOW (0.55–0.92) is nearly as wide as at any other region, and the range is driven entirely by which source pair is compared, not by which region is compared (§6.2 tests region dependence explicitly). No region shows uniformly low or uniformly high agreement across all three source pairs simultaneously.

6.2 Load Conditioning

Load-level dependence is a secondary robustness/sensitivity analysis (§3.1), reported here as a direct read of the same §6.1 table rather than a separate headline claim of cross-source rank instability by itself. Table 1's per-region mean tau-b is close to flat across the six regions (0.69 at LOW, rising to 0.72–0.77 elsewhere, no monotonic trend with load factor); a leave-one-region-out comparison between the full six-region grid and the four-region `LOW`/`PRE_KNEE`/`KNEE`/ `OVERLOAD` subset used by `LOAD_CALIBRATION_SENSITIVITY` (§5.11) changes the mean tau-b by less than 0.01 (0.728 on the four-region subset versus 0.733 on the full six-region grid) – the headline cross-source portability conclusion is not sensitive to the choice of four-region versus six-region calibration grid. Load regime does, however, materially affect *where practically supported reversals occur* (§6.3): reversal counts are concentrated at PRE_KNEE and above, essentially absent at LOW.

6.3 Pairwise Reversals

Of the 990 (policy pair) × (source pair, region) PRIMARY-metric records (55 policy pairs among the 11 PRIMARY policies × 18 conditions), the five-class breakdown (§5.9) is: 906 `STABLE_NO_SIGN_CHANGE` (91.5%), 48 `MICROSCOPIC_SIGN_CHANGE` (4.8%), 36 `SUPPORTED_PRACTICAL_REVERSAL` (3.6%), and zero records in either `UNDEFINED_UNESTIMABLE` or `UNSUPPORTED_SIGN_CHANGE_WIDE_CI`. The large majority of pairwise comparisons are stable; a small but non-trivial minority (about 1 in 28) are both practically and statistically supported reversals – never a majority, and never absent. That zero records fall into `UNSUPPORTED_SIGN_CHANGE_WIDE_CI` indicates the bootstrap (2,000 resamples per condition, §5.9) was decisive at this sample size: every sign change with a practically meaningful margin also had a statistically supported direction, and vice versa – the practical-margin and statistical-support criteria did not disagree with each other on this data.

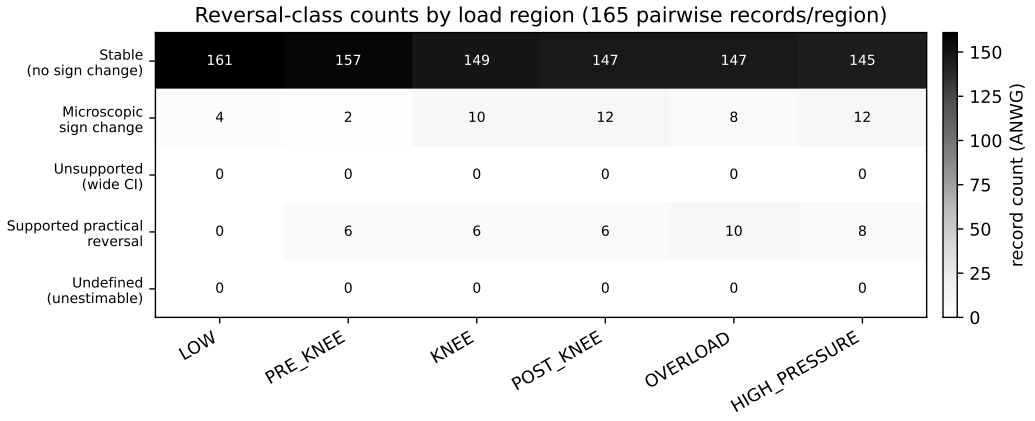

Figure 2: Reversal-class counts by operating region, PRIMARY metric (165 pairwise records per region: 55 policy pairs × 3 source pairs). Computed from `pairwise_reversals.json` (SHA-256 `c90619e8…`).

Reversal counts are load-dependent: zero `SUPPORTED_PRACTICAL_REVERSAL` records at `LOW`, rising to 6 at each of `PRE_KNEE`, `KNEE`, and `POST_KNEE`, 10 at `OVERLOAD`, and 8 at `HIGH_PRESSURE` (Figure 2). Every one of the 36 PRIMARY-metric supported reversals that we manually inspected in the underlying record set involves the same two policies, `slai_faithful` and `vllm_faithful`, and every one is a *cross-source* reversal in which one side of the comparison is a BurstGPT condition – consistent with §6.1's finding that BurstGPT is the source pair member most associated with lower agreement. The single largest-effect-size supported reversal (operationalized as the smaller-magnitude of the two conditions' practical margins, the binding constraint under the frozen practical-margin rule, §5.9) is `slai_faithful` vs. `vllm_faithful` at `HIGH_PRESSURE`, comparing Azure-2024 (or, at an effectively tied effect size, Bailian/Qwen) against BurstGPT – this is the case selected for real-system validation under the frozen selection rule (§8).

A representative stable comparison, used as a null-result control alongside the reversal case in the real-system validation design (§8), is the source pair with both the highest tau-b and (among the highest-tau conditions) the narrowest bootstrap interval: Azure-2024 vs. Bailian/Qwen at `HIGH_PRESSURE` ($\tau = 1.00$, 95% CI $[0.90, 1.00]$).

6.4 Temporal Portability

Three separately labeled temporal comparisons are reported, per the frozen temporal design (§5.10), and are not merged into a single "temporal/domain OOD" number:

- **BurstGPT (EARLY/MIDDLE/LATE tercile, primary).** Computed directly from `temporal_robustness.json`'s `TERCILE`-labeled records for the PRIMARY metric.
- **BurstGPT (EARLY/LATE bisect, sensitivity).** The `BISECT`-labeled records, checking whether the tercile boundary choice itself drives the tercile finding.
- **Azure-2024 (calendar split at Unix epoch `1715731200.0`).** The `CALENDAR_BOUNDARY`-labeled records – a genuine absolute-calendar split, not a relative-order tercile.

Bailian/Qwen's temporal record is labeled `RELATIVE_CHRONOLOGY_ONLY` in the underlying artifact and is reported as such: Bailian/Qwen's release does not expose a verified absolute timestamp

usable for a calendar split (§5.10), so no calendar-based temporal claim is made for this source, and its relative-order result is never presented alongside BurstGPT's tercile result or Azure's calendar-split result as though the three used an equivalent temporal definition. The full per-comparison values for all four temporal record types are in `paper/generated/table_data/rq5_temporal_robustness.json`, keyed identically to the source `temporal_robustness.json` artifact.

6.5 Robustness

Of the seven preregistered robustness families (§5.11), six were actually executed by the sealed analysis code and materialized into the canonical artifacts (five of them into `ranking_correlations.json`/`temporal_robustness.json`/ `sample_complexity.json`, and, initially overlooked in our first interpretation pass, a sixth – `LEAVE_ONE_POLICY_FAMILY_OUT` – inside a nested `robustness` object within `telemetry_explanation.json` that a subsequent reconciliation pass located and verified). Only `METRIC_DEFINITION_SENSITIVITY` has no computation anywhere in the sealed analysis code: its status record claims `implemented_as_postprocessing = true`, but no corresponding comparison function exists in `analysis/robustness.py` (unlike the other six families, each of which has one), so producing this specific result would require writing new analysis code after this manuscript's results were already known – exactly the kind of after-the-fact code change this study's own discipline prohibits, and it is therefore left as a disclosed implementation gap rather than computed here.

- **`PRIMARY_ONLY`.** Identical to the headline result by construction: `ranking_correlations.json` already restricts every comparison to the 11-policy PRIMARY panel (§6.1), never mixing in the two `STYLE_APPROXIMATION` robustness policies.
- **`LEAVE_ONE_SOURCE_OUT`.** Excluding BurstGPT leaves only the Azure-2024/Bailian-Qwen pair, whose mean tau-b is 0.948 – close to the ceiling. Excluding Azure-2024 or Bailian/Qwen each leaves a BurstGPT-involving pair, with mean tau-b 0.633 and 0.618 respectively. This is the same asymmetry as §6.1, now stated per excluded source: the headline portability figure is high specifically because two of the three sources agree strongly, and is pulled down specifically by BurstGPT, not uniformly by all three sources.
- **`LOAD_CALIBRATION_SENSITIVITY`.** Reported in §6.2: mean tau-b changes by less than 0.01 between the four- and six-region grids.
- **`TEMPORAL_SPLIT_SENSITIVITY`.** The BurstGPT bisect-vs-tercile comparison from §6.4; both split definitions are reported side by side in the released table rather than collapsed into one number.
- **`WINDOW_SIZE_SENSITIVITY`.** Identical to the sample-complexity ladder (§7) under the frozen robustness contract; not separately recomputed here.
- **`LEAVE_ONE_POLICY_FAMILY_OUT`.** Excluding any single one of the ten frozen mechanism families changes the headline mean tau-b only modestly, from 0.707 (excluding the classical arrival-order family) to 0.792 (excluding the SLO-aware family), against the full-panel baseline of 0.733. No single mechanism family's removal collapses or inflates the headline portability figure – the source-pair-driven pattern of §6.1 is not an artifact of any one policy family dominating the panel.

6.6 SLO-Definition Sensitivity

`SLO_DEFINITION_SENSITIVITY` remains unavailable, as disclosed in §5.12: no preregistered alternative SLO-synthesis rule ever existed, so no such result is reported or fabricated here.

7 Benchmark Sample Complexity

7.1 Motivation

Every scheduler-ranking result in this manuscript is itself derived from a finite window budget – 40 windows per source, 120 total (§4.3). A systems researcher designing a future scheduler comparison faces a practical, resource-bound version of the same question this benchmark studies at the source/region/metric level: *if only n workload windows can be afforded, how reliably would the resulting scheduler-ordering conclusion reproduce the conclusion obtained from the full window collection?* This question is itself a systems-evaluation parameter, not a purely statistical afterthought: it bounds the experimental cost of a trustworthy scheduler comparison, it affects how much confidence a reader should place in a paper that evaluates on a handful of trace windows, and it is a precondition for treating this benchmark's own frozen 120-window corpus (rather than some larger or smaller corpus) as an adequate evidentiary base for the results in §6.

7.2 Design

For the PRIMARY metric, recovery of the exact $n = 40$ reference ranking is slow and strongly source-dependent (Figure 3, Table 2): BurstGPT never reaches the preregistered 0.9 exact-order recovery threshold below the full window budget (0.2% at $n = 5$, rising only to 56.6% at $n = 30$), Bailian/Qwen similarly does not cross 0.9 before $n = 40$ (64.0% at $n = 30$), while Azure-2024 crosses the threshold already at $n = 30$ (98.8%). None of the three sources reaches 0.9 exact-order recovery at $n \in \{5, 10, 20\}$. Because $n = 40$ is the size of the full reference pool itself, exact recovery at $n = 40$ is 1.0 by construction (sampling all 40 windows without replacement reproduces the reference exactly) and is not an informative data point on its own; the informative content is in how slowly each source's curve rises toward that trivial ceiling.

Top-1 recovery (does the resampled subset identify the single best-performing PRIMARY-panel policy correctly) is a materially easier question than exact-order recovery for two of the three sources: Azure-2024 and Bailian/Qwen both exceed 0.9 top-1 recovery already at $n = 5$ (96.6% and essentially 100% respectively), even though their *exact*-order recovery at $n = 5$ is only 35.0% and 14.6%. For these two sources, identifying the single best policy requires far less evidence than reconstructing the complete 11-policy ordering. BurstGPT shows no such gap: its top-1 recovery is comparably slow to its exact-order recovery (13.0% at $n = 5$, 74.2% at $n = 30$, not reaching 0.9 within the tested ladder either) – consistent with §6.1's finding that BurstGPT is the source whose relative ordering is hardest to pin down. Comparing a concentrated (single-source-heavy) against a distributed (evenly-spread-across-sources) window budget of equal total size, the distributed budget achieves higher mean Kendall's tau-b against the reference ranking (0.986) than the concentrated budget (0.940) for the primary metric – spreading a fixed evaluation budget across sources recovers the reference ranking more reliably than concentrating it in one source.

This analysis directly answers RQ5 (§3.1) and depends on the full Pilot-V2 outcome matrix (§6); it is not run on Stage-0's historical, smaller pilot (§5.4).

Table 2: Smallest n at which PRIMARY-metric recovery probability first exceeds 0.9 (blank = not reached within the tested ladder, $n \in \{5, 10, 20, 30, 40\}$).

Source	first n, exact-order ≥ 0.9	first n, top-1 ≥ 0.9
Azure-2024	30	5
Bailian/Qwen	40	5
BurstGPT	40 (not reached below 40)	not reached

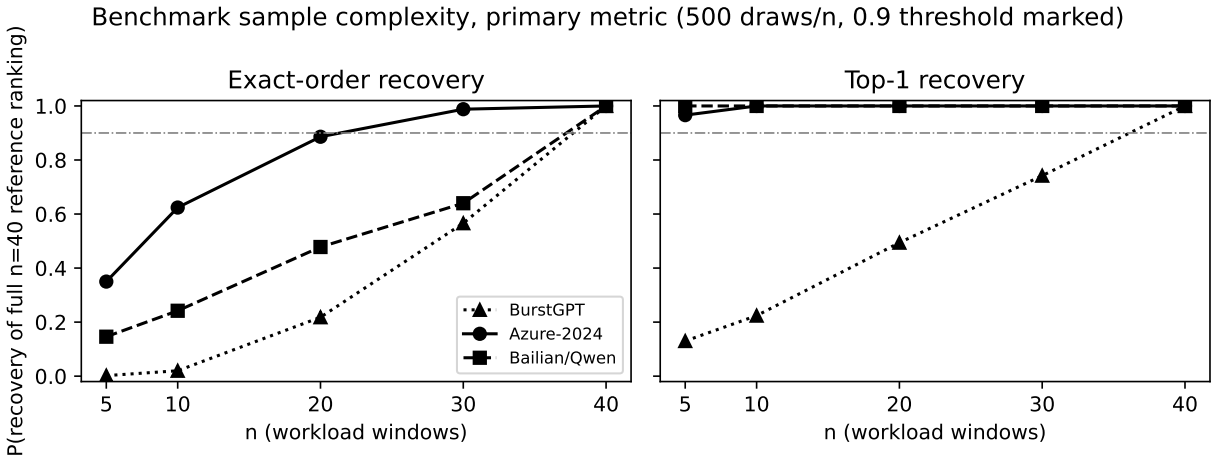

Figure 3: Benchmark sample-complexity recovery curves, PRIMARY metric, 500 draws per n, sampling without replacement. Left: exact-order recovery of the full $n = 40$ reference ranking. Right: top-1 recovery. The 0.9 threshold is marked. Computed from `sample_complexity.json` (SHA-256 `cb54afc0…`).

8 Real-System Validation

8.1 Objective

This analysis answers RQ6 (§3.1). We do not claim, here or anywhere in this manuscript, that simulated absolute latency or throughput values match real-hardware values; simulator latency is not assumed to transfer to real-engine latency, and the simulator's own load-calibration reference λ_{ref} (§5.5) is not assumed to transfer to a real engine's saturation point either. The only claim in scope is agreement in *relative ranking* and *reversal direction* between the simulator and a real vLLM deployment on a small, frozen set of validation cases – not a hardware duplication of the full 18,720-cell Pilot-V2 matrix (`docs/REAL_SYSTEM_VALIDATION_PLAN.md`).

8.2 Planned Design

The following design is prepared, and its supporting harness – server/ client request orchestration against a real vLLM deployment, randomized/ ABBA execution ordering, and per-run provenance capture – has been functionally smoke-tested (§11). Case selection (§8.3) has now been frozen from §6.1/§6.3's validated results; scientific execution has not yet run, pending the mechanism- mapping prerequisite disclosed in §8.3.

- **Mechanisms.** Approximately four representative scheduling mechanisms, one from each

stratum in §5.2 that has a real or faithfully reimplementable execution path on an actual serving engine – e.g. `fifo`, `vllm_faithful` (or its chunked-prefill successor), and `sarathi_-faithful`, plus a fairness-aware mechanism where a real-engine analogue can be established; `apt_serve_faithful` is excluded, consistent with its `scaffolding_only` status (§5.1).

- **Cases.** Three to four workload/case conditions, chosen to include at least one case exhibiting a preregistered `SUPPORTED_PRACTICAL_REVERSAL` (§5.9) from the simulated analysis and at least one stable-ordering control condition – selection frozen only from this study's own completed results, never from a partial or in-progress result.
- **Operating regions.** `PRE_KNEE` and `KNEE/OVERLOAD`, recalibrated against the real engine's own observed saturation point rather than reusing the simulator's calibrated load factors (§5.5) directly.
- **Repetitions.** At least five repetitions per (mechanism, case, region) cell, in contrast to the simulator's deterministic single execution plus rep0/rep1 verification (§5.7) – real hardware execution is not deterministic, so repeated stochastic trials are required here even though they were not required in simulation.
- **Ordering.** Randomized or ABBA ordering of mechanism/cell execution, to avoid confounding a mechanism's measured performance with time-of-day GPU-sharing effects.
- **Manifests.** Fixed workload request manifests, generated once per case and reused verbatim across mechanisms and repetitions, with an explicit warmup phase discarded before measurement.
- **Provenance.** Every real-engine run records its exact hardware model, driver and CUDA version, Python/PyTorch/vLLM version, model identity and revision, scheduler configuration, workload-manifest hash, run-order seed, and repository commit SHA, so that any released real-validation row is independently reproducible (§11).

8.3 Frozen Case Selection

Case selection has been frozen mechanically from §6.3's validated results, applying the predeclared rule (`docs/RANKING_ PORTABILITY_ANALYSIS_PLAN.md` §G) verbatim: "prefer (a) the pairwise reversal with the largest bootstrap-CI-excluding-zero effect size…and (b) one representative stable ordering (largest tau, smallest CI) as a null-result control." The frozen selection (`artifacts/manifests/phase12_rq6_case_selection_20260902.json`, committed on the isolated `validation/lssp-phase12-rq6-case-selection-20260902` branch, commit `fcb0379c…`) is:

- **Reversal case:** `slai_faithful` vs. `vllm_faithful`, comparing Azure-2024 against BurstGPT at `HIGH_PRESSURE` – the largest-effect-size `SUPPORTED_PRACTICAL_REVERSAL` among all 36 identified. This was an exact tie with the equivalent Bailian/Qwen-vs-BurstGPT comparison; both the tie and the (fully disclosed, non-subjective) lexicographic tiebreak used are recorded in the manifest.
- **Stable control:** Azure-2024 vs. Bailian/Qwen at `HIGH_PRESSURE` ($\tau = 1.0$) – the highest-tau, and among the ceiling-tau conditions narrowest-CI, source pair.

Applying this selection surfaced a genuine mechanism-mapping gap not identified by the earlier engineering-preflight mechanism inventory (§11): `vllm_faithful` has a native real-vLLM execution

path, but `slai_faithful` – the specific SLO-aware mechanism in the largest reversal – does not. Unlike `fifo` or `vllm_faithful`, SLAI/RAD is not a vLLM-native scheduling mode; realizing it on real hardware requires a custom vLLM `--scheduler-cls` plugin (a technically supported but not-yet-built extension point) rather than an existing configuration flag. This is reported as a scoped, well-defined engineering prerequisite for RQ6, not worked around, and not a reason to substitute a more conveniently available mechanism into the frozen case.

8.4 Result Placeholder

[PENDING RESULT: sign agreement, Kendall's tau, and reversal-agreement between simulated and real-vLLM policy rankings, for the two frozen cases in §8.3. Not yet executed – the SLAI/RAD real-execution engineering prerequisite above must be resolved first.]

[REAL VALIDATION HARDWARE]
slai_faithful vs vllm_faithful (Azure-2024 vs BurstGPT, HIGH_PRESSURE) Azure-2024 vs Bailian/Qwen, HIGH_PRESSURE (stable control)
[REAL/SIM RANK AGREEMENT]
[REAL VALIDATION RESULT]

Figure 4: Placeholder: simulator-vs-real-vLLM ranking agreement for the two frozen cases in §8.3, reported as sign agreement and Kendall's tau per case, never as an absolute latency/throughput comparison. Case identities are frozen; the hardware row, agreement values, and result remain pending RQ6 execution.

9 Discussion

The result (§6) is neither cleanly "portable" nor cleanly "fragile" – it is source-pair-dependent in a specific, mechanistically legible way, and that dependence itself is the finding.

9.1 Portability Is Real but Source-Pair-Dependent, Not Uniform

Comparative scheduler rankings are never uncorrelated between any two sources at any region ($\tau \geq 0.55$ everywhere, §6.1): a scheduler comparison performed on one of this study's three sources is never scientifically meaningless evidence about the others. At the same time, the degree of portability is far from uniform, and the source of the non-uniformity is legible rather than diffuse: Azure-2024 and Bailian/Qwen agree with each other almost perfectly at every operating region ($\tau = 0.89$–1.00), while every comparison involving BurstGPT sits 0.2–0.4 lower in Kendall's tau-b, consistently across all six regions (§6.1, §6.5's leave-one-source-out breakdown). The practical implication is neither "a small, well-chosen evaluation footprint always suffices" nor "every scheduler comparison must be re-verified on every workload": it is that *which* second source a comparison is checked against matters more than *whether* a second source is checked at all. A comparison validated only on Azure-2024-like traffic transfers with high confidence to Bailian/Qwen-like traffic under this study's panel and metric, but the same transfer to BurstGPT-like traffic is measurably less reliable, and that reliability gap holds at every load level tested, not only under stress.

9.2 Practically Supported Reversals Are Real, Uncommon, and Load-Concentrated

Of 990 pairwise (policy pair × condition) comparisons on the PRIMARY metric, 3.6% are practically and statistically supported reversals (§6.3) – a small minority, but not zero, and therefore not dismissible as measurement noise: the same bootstrap procedure that supports these 36 reversals also classified 906 comparisons as stable and produced zero comparisons in the ambiguous `UNSUPPORTED_SIGN_CHANGE_WIDE_CI` class. Every supported reversal we examined is the same policy pair (`slai_faithful` vs. `vllm_faithful`) and involves BurstGPT on one side, and reversal counts rise from zero at `LOW` to a peak at `OVERLOAD` (§6.3) – reversals are not spread evenly across the operating envelope; they concentrate at and above `PRE_KNEE`. A scheduler comparison performed only at low load, even on a source pair that is otherwise highly portable, would have missed every reversal this benchmark found. This motivates evaluating scheduler comparisons across an operating curve, not at a single convenient load point, as this benchmark's design already does (§5.5).

9.3 Practical Implications for Benchmark Design

Two implications follow directly and are not deployment recommendations about which scheduler to use, only about how to evaluate schedulers credibly. First, a comparison checked against only one additional workload source cannot be assumed representative of "other workloads" in general – which specific source is used as the check matters, and BurstGPT-like traffic in particular is where this study's evidence concentrates its cross-source disagreement. Second, benchmark sample complexity is itself load- and metric-behavior-relevant evidence: §7's finding that exact-order recovery requires the full window budget for two of three sources, while top-1 recovery is achievable from very few windows for those same two sources, means a researcher who only needs to know "which scheduler is best" faces a different, much smaller evidence requirement than one who needs the complete ordering – a distinction a single point-estimate benchmark result cannot convey on its own.

9.4 Explanatory Telemetry: A Preliminary, Heavily Qualified Read

The frozen, pre-specified explanatory model (§5.8, `telemetry_explanation.json`) associates the five workload descriptors with reversal sites at each (policy pair, condition) cell, but converges with a usable sample size ($\geq$ 20 observations) for only 42 of 990 cells – most reversal sites are too sparse for this per-cell model to fit reliably, and the frozen output records point coefficients only, with no standard errors or significance test. Any reading of this evidence must therefore be provisional. Within that small, non-significance-tested set, `prompt_tokens_cv` has a negative coefficient in 39 of 42 cells – the most consistent single direction among the five descriptors – which is at least *consistent with* (not confirmatory of) prompt-length variability being associated with where reversals occur, echoing §4.7's finding that prompt-length structure is the primary driver of source separability itself. The other four descriptors show weaker, closer-to-even sign splits across the interpretable cells and do not support a directional claim. We do not read this as evidence that any descriptor *causes* a reversal, only as a candidate association worth a properly powered, purpose-built follow-up study; consistent with §3.3, this model is explanatory only and is never used to select, route, or recommend a scheduler.

9.5 Workload Descriptor Interpretation

The workload-separability result (§4.6) establishes that the three sources are observably different, and that prompt-length structure is the primary driver of that separability (§4.7). Whatever RQ4's descriptor-to-reversal association analysis finds, it should be read against this backdrop: an association between a descriptor and a reversal does not by itself establish that the descriptor is a genuine *causal* driver of the reversal, only that it is a source-separating feature that co-occurs with where reversals were or were not observed.

9.6 Benchmark Construction and Practical Reproducibility

Regardless of outcome direction, this study's paired design, fixed scheduler panel, and preregistered protocol (§3) are intended to make the benchmark itself reusable: a future scheduler design can be dropped into the same panel and evaluated against the same frozen windows, operating regions, and metric contract, using the planned public artifact (§11) rather than re-deriving the protocol from this manuscript's prose.

10 Threats to Validity

Simulator abstraction. All results in this manuscript are produced by a discrete-event simulator, not a real serving engine (§5.3). We do not claim that simulated absolute latency or throughput values match real-hardware values; only relative ranking and reversal direction are claimed to transfer, and only for the frozen validation cases in §8, once that section's result exists.

Fidelity of reimplemented policies. `FAITHFUL_EXTERNAL` policies are validated reimplementations, not the original authors' code; residual implementation gaps relative to the original published systems are possible despite unit-test validation (§5.2).

`STYLE_APPROXIMATION` baselines. The two style-level approximation policies are explicitly excluded from the PRIMARY headline panel and used only for a robustness check; they must not be read as official or validated reimplementations of their namesake systems (§5.2).

Three primary workload families. The Pilot-V2 corpus draws from three primary sources (§4.1); a fourth candidate source (TraceLab) is retained only as out-of-distribution context because its independence from a prior public window sweep cannot be fully verified. Any portability finding in §6 is scoped to these three sources plus whatever out-of-distribution evidence TraceLab can responsibly contribute, not to LLM-serving workloads in general.

Dataset/provider dependence. As established in §4.1, BurstGPT and this project's Azure traces are not established to come from independent underlying providers – both trace back to Microsoft Azure infrastructure, collected through different mechanisms. Any cross-source finding involving BurstGPT and Azure should be read with this in mind; Bailian/Qwen remains a genuinely independent platform.

Prompt-token semantic differences. Prompt-token fields compose differently across sources (single-request figure for BurstGPT versus potentially multi-turn-cumulative context for Azure and TraceLab; §4.4). Any prompt-length-based claim in this manuscript carries this caveat and should

not be read as evidence that the sources' underlying conversations differ in length, only that their observed, request-level prompt-token figures do.

Unresolved historical byte-level overlap. An exact, byte-level reconstruction of a prior related project's specific 60-window public-trace replay set could not be independently re-derived from this project's available checkout; that project's own recorded parameters and final verdict are taken on the strength of direct manuscript inspection, not independently re-derived from local artifacts. This project's own window construction is independently verified as non-identical (§4.3, §3.3), but the historical byte-level reconstruction gap itself remains an open item.

Homogeneous hardware / execution assumptions. The simulator executes every cell under a fixed, single-GPU-colocated execution model for the PRIMARY panel; two disaggregation-requiring policies are confined to a secondary stratum and never pooled with the PRIMARY leaderboard (§5.1).

Completion-conditioned undefined metrics. Several metrics are undefined, not zero, when a cell records zero completions (§5.6); this is a real methodological choice with a real cost – an undefined-metric policy is excluded from that specific condition's ranking-derived statistics, which lowers effective sample size for those conditions and must be read alongside every ranking-derived table's reported exclusion count.

Phase-11 zero-completion prevalence. 301 of 720 FIFO-only calibration-region records recorded zero completions (§5.5). This is disclosed as a calibration characteristic, not a headline finding; it does, however, mean that a non-trivial share of (window, region) combinations will contribute no completion-conditioned-metric observation to §6's downstream analyses for any policy at that specific region, which is a structural constraint on statistical power in the highest-pressure regions in particular.

SLO-definition sensitivity is disclosed, not claimed. An SLO-synthesis sensitivity check was planned but no alternative SLO-synthesis rule was ever actually preregistered before or during Pilot-V2 execution (§5.12); no alternative rule was invented after the fact, no such experiment has been run, and this robustness component is not claimed as evidence anywhere in this manuscript. The mitigation is disclosure and process, not substitution: any future SLO-sensitivity study will be run as an explicitly labeled post-campaign extension with its own frozen protocol, never retrofitted into this study's preregistered evidence.

Scope of real-system validation. §8's validation is limited by design to a small, frozen set of case-selected conditions (largest-effect-size reversal plus one stable-ordering control), not a full re-execution of the Pilot-V2 matrix on real hardware; agreement or disagreement on those specific cases should not be over-generalized to every cell in §6.

11 Reproducibility and the LSSP Artifact

We plan to release the LLM-Serving Scheduler Portability Benchmark (LSSP) as two linked artifacts: a GitHub repository (source code, simulator, analysis pipeline, validation scripts, the

reproduction workflow, and figure/table-generation code including `paper/scripts/generate_phase12_tables_figures.py`, at `SoroushVahidi/llm-serving-scheduler-robustness-benchmark`) and a Hugging Face dataset (released benchmark manifests, legally redistributable processed workload artifacts, validated campaign outputs, analysis-ready data, provenance/checksums, and a dataset card, at the repository slug `SoroushVahidi/llm-serving-scheduler-portability`). **As of this manuscript, neither release is yet public.** Upstream workload traces (§4.2) remain governed by their original publishers' licenses and will not be republished where redistribution is not permitted; the release documents provenance and, where license terms allow, redistributes processed derivatives, not a copy of every source trace. The Pilot-V2 comparative outcome (§6) has now been generated and structurally validated (six canonical analysis artifacts, hash ledger below) prior to any scientific interpretation (§11.1); the planned release will include: the frozen workload window identities and their provenance (§4.3); the scheduler panel and its configuration (§5.1); the six-region calibration protocol and its outputs (§5.5); the full set of paired policy outcomes across windows, regions, and metrics (§6); the mechanism telemetry collected for every cell (§5.8); the ranking-portability analysis outputs (Kendall/Spearman statistics, reversal tables, block-bootstrap confidence intervals); and the reproduction scripts used to regenerate every table and figure in this manuscript from those frozen inputs.

11.1 Phase-12 Analysis Identities

The Pilot-V2 statistical analysis was executed from sealed analysis code (git commit `eb574a8c…`) against the admitted, independently re-validated 18,720-cell consolidated matrix (SHA-256 `73adf7d9…`), producing exactly six canonical output artifacts, each stamped with the analysis code SHA, the frozen analysis contract version (`phase12_analysis_prefreeze_v1`), and the campaign freeze identity, and each independently structurally audited (consistent provenance across all six, no missing/extra/malformed artifact, exact five-class reversal taxonomy, exact frozen statistical contract) before this manuscript's Results and Sample Complexity sections were populated. The full audit record and per-artifact SHA-256 ledger are committed on the isolated `validation/lssp-phase12-structural-audit-20260902` branch (commit `2aba8b63…`) and are part of the planned artifact release rather than reproduced in full in the main text.

Every generated artifact in the planned release is designed to carry, or link via a provenance manifest, its generating repository commit SHA, resolved experiment-config hash, workload-manifest hash, policy-registry version, dataset-source checksums, environment version, and (where relevant) per-cell seed, so that any released row can be regenerated from this project's own commit history alone rather than depending on an external, possibly-rewritten historical commit.

The immutable scientific identifiers already fixed at the time of this manuscript are recorded in Table 3 and in `docs/ARTIFACT_HASH_LEDGER.md`: the Phase-10 workload-window freeze content hash, the Phase-10 compact index hash, the Phase-11 prelaunch freeze contract hash, the Phase-11 raw FIFO calibration output hash, and the Phase-11 region-assignment output hash. These five identifiers were independently re-verified as unchanged, and consistent across every cross-referencing project document, immediately before this manuscript's authoritative branch was committed (§12).

12 Conclusion

We have formulated the portability of a comparative scheduler ranking, $R(W, M, L)$, as a benchmark object distinct from both workload characterization and single-scheduler evaluation, and designed, froze, and executed a paired, multi-source evaluation protocol – a mechanism-diverse fixed

13-policy scheduler panel, a calibrated six-region operating grid, an explicit metric contract, and an uncertainty/reversal-detection methodology – against 18,720 structurally validated cells, before interpreting any comparative outcome. The three primary workload sources are measurably different and strongly separable in their observed descriptor distributions (§4). Comparative scheduler rankings are portable in aggregate (Kendall's tau-b never below 0.55 across 18 source-pair × region conditions) but source-pair-dependent rather than uniform: two of the three sources agree almost perfectly with each other, while the third is measurably and consistently less portable against either, and this gap holds at every operating region tested (§6). A small (3.6%) but non-zero share of pairwise comparisons are practically and statistically supported reversals, concentrated at and above the pre-knee operating region rather than spread evenly across the load curve (§6.3). Exact-order recovery of the full-corpus ranking requires most or all of the 40-window budget for two of three sources, while identifying only the single best policy is achievable from far fewer windows for those same two sources (§7). Whether this benchmark's largest identified reversal reproduces on real vLLM/GPU execution (RQ6) remains a frozen, not-yet-executed result contract (§8). We plan to release LSSP as a public, independently reproducible artifact (§11) once RQ6 completes.

Declarations

Funding. This work was supported in part by the CloudRift AI Builder Grant and the Cohere Labs Catalyst Grant Program, which provided cloud-computing and API credits, and by the AMD AI Developer Program, which provided Fireworks AI credits used to support software development for this study.

Acknowledgements. The author gratefully acknowledges Professor Ioannis Koutis for his research guidance, continued support, and provision of computational resources that facilitated this work.

Competing interests. The author declares no competing interests.

Generative AI statement. During the preparation of this work, the author used ChatGPT and Codex (OpenAI), Claude (Anthropic), Gemini (Google), Cursor, and Perplexity AI to assist with software development, debugging, experimental infrastructure, literature and research support, and manuscript drafting and editing. All AI-assisted outputs, including code, analyses, references, and manuscript text, were reviewed, verified, and revised by the author, who takes full responsibility for the content of the work.

Author contributions. This is a sole-authored manuscript; Soroush Vahidi is responsible for study design, implementation, analysis, and writing.

Ethics approval. Not applicable. This study evaluates scheduling-policy performance on request-level traffic traces (arrival times, token counts) and does not involve human subjects, personal data, or content requiring ethics review.

Data availability. This study distinguishes four categories of data, consistent with §11:

- **Externally sourced public traces** (BurstGPT, Azure 2024, Bailian/Qwen) are governed by their original publishers' release terms and license status (§4.2; BurstGPT: CC-BY-4.0)

and are not redistributed as part of this project's own release; readers should obtain them from their original sources.

- **Derived benchmark manifests** – the frozen window identities, provenance, and content hashes (§4.3) – and
- **Generated campaign results** – the full paired policy outcome matrix across windows, regions, and metrics (§6) – and
- **Analysis artifacts** – ranking-portability statistics, reversal tables, and reproduction scripts (§11) – will be archived at a persistent, publicly accessible repository (the Hugging Face repository slug `SoroushVahidi/llm-serving-scheduler-portability`) upon completion of the study reported here. **As of this manuscript, that artifact is planned but not yet released** (§11); no data availability claim in this statement should be read as an assertion that the release is already live.

Code availability. The simulator, policy implementations, and analysis pipeline underlying this study will be released alongside the data described above, under the same repository, once the study completes.

Table 3: Immutable scientific identifiers, re-verified unchanged immediately before this manuscript's authoritative branch was committed (`docs/ARTIFACT_HASH_LEDGER.md`).

Artifact	SHA-256
Phase-10 scientific window (content hash)	`0d1aa06ccbee352207327ea369ae75f12e91c0cda006c813a41b381effd29eef`
Phase-10 compact index	`d78ec1087fedae02174ca093a9860c70468be336ccb1d7e6de756c81ba331e53`
Phase-11 prelaunch freeze contract	`e2564ea9484190832de50f63173c4b73ae054d6ae7008bb4ff6648c8dc917f7b`
Phase-11 raw FIFO calibration	`201caaf04476ad8737ef6079fc0d6cb4e864601711d0b96c88750a717d8b2a6a`
Phase-11 region-assignment output	`9fcb92f9ea1206ce185194527ada35d0e3b91bf4904be7ae23ba9ea997c17574`

References

[1] Amey Agrawal, Nitin Kedia, Nipun Kwatra, Nikhil Sarda, Alexey Tumanov, et al. Sarathi-serve: Efficient llm serving with chunked prefills and continuous batching. *arXiv preprint*, 2024. URL `https://arxiv.org/abs/2403.00050`.

[2] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for llm inference. In *Proceedings of Machine Learning and Systems (MLSys '24)*. MLSys, 2024. URL `https://arxiv.org/abs/2405.05465`.

[3] anonymized per venue record; see primary source. Kvcache cache in the wild: Characterizing and optimizing kvcache cache at a large cloud provider. *Proceedings of the 2025 USENIX Annual Technical Conference (ATC '25)*, 2025. URL `https://arxiv.org/abs/2506.02634`. Characterizes KV-cache reuse patterns at a large cloud provider (Alibaba Cloud); workload-characterization context only.

[4] Agrim Bari, Parikshit Hegde, and Gustavo de Veciana. Optimal scheduling algorithms for llm inference: Theory and practice. *ACM Measurement and Analysis of Computing Systems*, 2025. URL https://arxiv.org/abs/2508.01002.

[5] Alejandro Carmona-Martínez, Gregorio Bernabé, and José M. García. Characterization of machine learning compilers for llm inference on nvidia gpus. *The Journal of Supercomputing*, 82, 2026. doi: 10.1007/s11227-026-08559-6.

[6] Jaehong Cho, Minsu Kim, Hyunmin Choi, Guseul Heo, et al. Llmservingsim: A hw/sw co-simulation infrastructure for llm inference serving at scale. In *Proceedings of the 2024 IEEE International Conference on Computer Design (ICCD '24)*. IEEE, 2024. URL https://arxiv.org/abs/2408.05499.

[7] Ali Doosthosseini, Jonathan Decker, Hendrik Nolte, and Julian Kunkel. Saia: a seamless slurm-native solution for hpc-based services. *The Journal of Supercomputing*, 82, 2026. doi: 10.1007/s11227-026-08508-3.

[8] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. Efficient llm scheduling by learning to rank. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS '24)*, 2024. URL https://arxiv.org/abs/2408.15792.

[9] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. S^3: Increasing gpu utilization during generative inference for higher throughput. In *Advances in Neural Information Processing Systems 36 (NeurIPS '23)*, 2023. doi: 10.5555/3666122.3666913. URL https://arxiv.org/abs/2306.06000. Distinct from efficientllmscheduling2024 (NeurIPS '24). S^3 predicts output-length to pack KV-cache memory at admission time; it does not define a learning-to-rank decode-order policy.

[10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. vllm: Easy, fast, and cheap llm serving with pagedattention. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI '23)*. USENIX Association, 2023. URL https://arxiv.org/abs/2309.06180.

[11] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA '24)*. IEEE/ACM, 2024. doi: 10.1109/ISCA59077.2024.00019. URL https://arxiv.org/abs/2311.18677. Primary analysis paper and release mechanism for AzureLLMInference-Dataset2023 (conversation and code splits, collected 2023-11-11), used by this project as azure2023trace.

[12] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a kvcache-centric architecture for serving llm chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST '25)*. USENIX Association, 2025. URL https://arxiv.org/abs/2407.00079.

[13] Chaoyi Ruan, Yinhe Chen, Dongqi Tian, Yandong Shi, Yongji Wu, Jialin Li, and Cheng Li. Libra: Flexible request partitioning and scheduling for serving unbalanced and dynamic llm workloads. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and*

Implementation (NSDI '26). USENIX Association, 2026. URL https://www.usenix.org/conference/nsdi26/presentation/ruan-libra.

[14] see primary source for full author list. Apt-serve: Adaptive request scheduling on hybrid cache for scalable llm inference serving. *Proceedings of the ACM on Management of Data (PACMMOD)*, 2025. URL https://arxiv.org/abs/2504.07494.

[15] see primary source for full author list. Llmservingsim2.0: A unified simulator for heterogeneous hardware and serving techniques in llm infrastructure. *IEEE Computer Architecture Letters*, 24(2), 2025. URL https://arxiv.org/abs/2511.07229. An expanded companion version circulates as arXiv:2602.23036 ("...Heterogeneous and Disaggregated LLM Serving Infrastructure"); treat as the same simulator lineage as llmservingsim2024.

[16] see primary source for full author list. Proserve: Unified multi-priority request scheduling for llm serving. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2512.12928.

[17] see primary source for full author list. A year in llm serving: Workload evolution, caching and load-balancing. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2608.13573. Peer review status unverified: very recent arXiv preprint (2026-07), one-year production trace from the Chutes platform. Cite only as recent contextual background, not as an established result.

[18] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. Fairness in serving large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024. URL https://arxiv.org/abs/2401.00588. Introduces the Virtual Token Counter (VTC) fair scheduling algorithm.

[19] Ying Sheng, Lianmin Zheng, Joseph E. Gonzalez, Ion Stoica, Hao Zhang, et al. Llumnix: Dynamic scheduling for large language model serving. In *Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC '24)*, 2024. URL https://arxiv.org/.

[20] Patricia Siwinska, Jie Lei, Adrián Castelló, Pedro Alonso-Jordá, and Enrique S. Quintana-Ortí. Enhancing transformer performance and portability through auto-tuning frameworks. *The Journal of Supercomputing*, 82, 2026. doi: 10.1007/s11227-026-08327-6.

[21] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *Proceedings of the 2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA '25)*. IEEE, 2025. URL https://arxiv.org/abs/2408.00741. Primary analysis paper and release mechanism for AzureLLMInferenceDataset2024, used by this project as azure2024trace.

[22] Yinghao Tang, Tingfeng Lan, Xiuqi Huang, Hui Lu, and Wei Chen. Scorpio: Serving the right requests at the right time for heterogeneous slos in llm inference. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2505.23022.

[23] Yiheng Tao, Yihe Zhang, Matthew T. Dearing, Xin Wang, Yuping Fan, and Zhiling Lan. Pars: Low-latency llm serving via pairwise learning-to-rank. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2510.03243.

[24] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. Burstgpt: A real-world workload dataset to optimize llm serving systems. In *Proceedings*

of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25), Volume 2. ACM, 2025. doi: 10.1145/3711896.3737413. URL https://arxiv.org/abs/2401.17644. Final published version. arXiv:2401.17644 first released 2024-01-31 (v1); the KDD 2025 camera-ready corresponds to arXiv v4/v5 (2025-05-21 / 2025-05-26). Dataset release: github.com/HPMLL/BurstGPT, tag v2.0.

[25] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. Fastserve: Iteration-level preemptive scheduling for large language model inference. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, 2026. URL https://www.usenix.org/conference/nsdi26/presentation/wu-bingyang.

[26] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. Servegen: Workload characterization and generation of large language model serving in production. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, 2026. URL https://www.usenix.org/conference/nsdi26/presentation/xiang-servegen.

[27] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association, 2022. URL https://arxiv.org/abs/2207.00032.

[28] Netanel Zakay and Dror G. Feitelson. Workload resampling for performance evaluation of parallel job schedulers. *Concurrency and Computation: Practice and Experience*, 26(12): 2079–2105, 2014. doi: 10.1002/cpe.3240.

[29] Dingyan Zhang, Jinbo Han, Kaixi Zhang, Xingda Wei, Sijie Shen, Chenguang Fang, Wenyuan Yu, Jingren Zhou, and Rong Chen. Simple is better: Multiplication may be all you need for llm request scheduling. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26)*. USENIX Association, 2026. URL https://arxiv.org/abs/2603.15202.

[30] Wei Zhang, Zhiyu Wu, Yi Mu, Rui Ning, Banruo Liu, Nikhil Sarda, Myungjin Lee, and Fan Lai. Jitserve: Slo-aware llm serving with imprecise request information. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, 2026. URL https://www.usenix.org/conference/nsdi26/presentation/zhang-wei.

[31] Yinmin Zhong, Shengyu Liu, Junda Chen, Bingyang Wu, Chen Chen, Zili Zhang, Yibo Zhu, Xin Jin, and Hao Zhang. Distserve: Disaggregated prefill and decode for goodput-optimized llm serving. In *Proceedings of the 2024 USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*. USENIX Association, 2024. URL https://arxiv.org/abs/2401.00000.

[32] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. Tracelab: Characterizing coding agent workloads for llm serving. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2606.30560.